

RAG Pipeline Technical Report: Retrieval-Driven QA with Switch Cues

NLP and IR Session

May 2026

1 Introduction

This report details the construction of a Retrieval-Augmented Generation (RAG) pipeline[cite: 6, 7]. The system follows a modular architecture: Corpus → Chunks → Embeddings → Indexing → Retrieval → Re-ranking → Generation[cite: 20, 21].

2 Checkpoint 1–3: Lexical vs. Dense Retrieval

Initially, we implemented **BM25 (Lexical Retrieval)** which relies on exact term overlap[cite: 33, 37]. We then transitioned to **Dense Retrieval** using vector embeddings to solve the "vocabulary mismatch" problem, where meanings match but words do not[cite: 41, 45, 46]. For scalability, we utilized **FAISS** for Approximate Nearest Neighbor (ANN) search[cite: 70, 79].

3 Checkpoint 4–5: Chunking and Grounded QA

Documents were split into chunks to ensure precise retrieval[cite: 25, 84]. We established a "Grounded QA" protocol where the model is legally contracted to answer only using the retrieved context, preventing hallucinations[cite: 96, 97, 109, 112].

4 Checkpoint 6: Failure Analysis & Improvement

We identified common failure modes, such as *Missing Evidence* and *Hallucination*[cite: 123, 129]. To improve the system, we implemented a refusal logic where the system says "I do not know" if the evidence is insufficient[cite: 114].

5 Checkpoint 7: Advanced Tools (Re-ranking & DistilBERT)

To enhance precision, we added a re-ranking stage using a stronger scorer and priority "Switch Cues"[cite: 141, 142].

5.1 Heuristic Re-ranking Implementation

The following function boosts scores based on priority terms such as "severe" or "allergic":

```
1 def rerank(query, retrieved_docs):
2     priority_terms = ["severe", "allergic", "rare"]
3     scored = []
4     for doc in retrieved_docs:
5         score = 0
6         text = doc["text"].lower()
7         for term in priority_terms:
8             if term in query.lower() and term in text:
9                 score += 3
10        score += len(set(query.lower().split()) & set(text.split()))
11        scored.append((score, doc))
12    return sorted(scored, key=lambda x: x[0], reverse=True)
```

5.2 DistilBERT Reader

We utilized **DistilBERT** (a compressed version of BERT) to extract exact answers from context[cite: 150, 151].

```
1 from transformers import pipeline
2 qa_pipeline = pipeline("question-answering",
3                         model="distilbert-base-cased-distilled-squad")
4
5 result = qa_pipeline(question="What are common side effects?",
6                      context="Common side effects include nausea.")
```

6 Conclusion

By integrating **LangChain** as the glue for our components, we successfully built a pipeline that is grounded, diagnosable, and resistant to common IR failures[cite: 159, 160].